



PROYECTO SOCIO FORMATIVO

Mini Sistema para la Gestión de una pequeña Biblioteca

Docente:

Ing. Jimmy Nataniel Requena Llorentty

Integrantes:

Jaquelin Priscila Bellido Duran

Douglas Flor Hernández

Carla Rachel Pedraza Ovaes

Luis Alberto Peloza Pedriel

Jhonny Pérez Bellido

Santa Cruz - Bolivia

INDICE

INTRODUCCIÓN	4
DEDICATORIA	5
AGRADECIMIENTOS	5
CAPITULO I	6
ANTECEDENTES	6
1.1 PLANTEAMIENTO DEL PROBLEMA	6
1.2 JUSTIFICACION	6
1.3 TIPO Y NIVEL DE INVESTIGACION	7
1.4 OBJETIVOS DEL PROYECTO	8
1.4.1 Objetivo General	8
1.4.2 Objetivos específicos	8
CAPITULO II	8
METODOLOGÍA DEL TRABAJO	8
2.1 ORGANIGRAMA DE LA ORGANIZACIÓN GRUPAL	9
2.2 DESCRIPCIÓN DEL ÁMBITO DE LA INVESTIGACIÓN	9
CAPITULO III	10
MARCO CONCEPTUAL	10
3.1 VISUAL STUDIO CODE (VSCODE)	11
3.2 HYPERTEXT MARKUP LANGUAGE	Error! Bookmark not defined.
3.3 CASCADING STYLE SHEETS	Error! Bookmark not defined.
3.4 PHP: Lenguaje de Programación para Desarrollo Web	Error! Bookmark not defined.
3.5 MySQL: Sistema de Gestión de Bases de Datos Relacionales	Error! Bookmark not defined.
3.6 GITHUB	Error! Bookmark not defined.
3.7 XAMPP: Entorno de Desarrollo para Servidores Web	Error! Bookmark not defined.
CAPITULO IV	Error! Bookmark not defined.
MODELADO DE LOS ARTEFACTOS DEL SISTEMA	Error! Bookmark not defined.



4.2 MODELADO ESTRUCTURAL	Error! Bookmark not defined.
CAPITULO V	16
PROTOTIPADO INICIAL	16
5.1 PROCESO DE DISEÑO EN VSCODE	17
CAPITULO VI	17
DESARROLLO DEL SISTEMA GESTION BIBLIOTECARIA	17
CAPITULO VII	28
CONCLUSIONES DEL PROYECTO	28
CAPITULO IX	30
RECOMENDACIONES GRUPALES	30
CAPITULO X	31
REFERENCIAS BIBLIOGRAFICAS	31

INTRODUCCIÓN

La gestión de bibliotecas ha experimentado una transformación significativa con la incorporación de tecnologías digitales que responden a la necesidad de mejorar la organización, el acceso y el control de los recursos bibliográficos. Desde los antiguos registros en papel hasta los modernos sistemas de gestión en línea, el uso de herramientas tecnológicas ha permitido optimizar procesos como el préstamo de libros, el registro de usuarios y la administración de inventarios de manera más eficiente y accesible.

En este contexto, el presente proyecto tiene como objetivo el desarrollo de un mini sistema web modular para la gestión de una pequeña biblioteca. Este sistema permitirá llevar un control ordenado de los libros disponibles, gestionar el préstamo y la devolución de ejemplares, registrar usuarios, y generar reportes de actividad bibliotecaria según rangos de fechas. Además, el sistema incluirá funcionalidades complementarias como la búsqueda de libros por título o autor y la posibilidad de extenderse hacia una Aplicación Web Progresiva (PWA) para mejorar la experiencia de los usuarios.

Desde el punto de vista técnico, la aplicación será desarrollada utilizando PHP para el backend y MySQL como sistema gestor de bases de datos, bajo una arquitectura Modelo-Vista-Controlador (MVC). El frontend se construirá con HTML5 y CSS, garantizando una interfaz clara e intuitiva. El sistema será alojado en un servidor en la nube, lo que permitirá su disponibilidad constante y escalabilidad.

Para asegurar una estructura organizada y bien documentada, se empleará el enfoque UML (Lenguaje Unificado de Modelado), incluyendo diagramas de casos de uso y diagramas de clases, así como diagramas de secuencia para las operaciones principales del sistema. Entre los diagramas contemplados se encuentran:

Caso de uso principal del sistema

Caso de uso para el préstamo y devolución de libros

Caso de uso para la gestión de usuarios

Diagrama de secuencia para el proceso de registro y préstamo

Este proyecto tiene como finalidad facilitar la administración de una biblioteca pequeña mediante una herramienta tecnológica sencilla pero eficaz, además de demostrar la aplicación de principios de programación orientada a objetos, diseño estructurado y desarrollo de servicios web para contextos educativos o comunitarios.

DEDICATORIA

Este proyecto está dedicado al Ingeniero Jimmy Nataniel Requena Llorentty, docente de la materia Programación III, por su compromiso, orientación y apoyo constante durante todo el proceso para el desarrollo de Mini Sistema para la Gestión de una pequeña Biblioteca. Su conocimiento y experiencia han sido fundamentales para guiarnos en cada etapa del diseño, modelado y programación del sistema, permitiéndonos fortalecer nuestras habilidades técnicas y nuestra comprensión de los principios del desarrollo. Gracias a su dedicación y valiosas enseñanzas, hemos logrado enfrentar y superar los desafíos del proyecto, consolidando una base sólida para nuestro crecimiento profesional.

AGRADECIMIENTOS

Expresamos nuestro más sincero agradecimiento al Ingeniero Jimmy Nataniel Requena Llorentty, por su paciencia, guía y exigencia académica, que nos motivaron a esforzarnos al máximo en cada aspecto del desarrollo del sistema. Su apoyo y retroalimentación han sido clave para garantizar la calidad y funcionalidad del proyecto.

A nuestras familias, por su apoyo incondicional, comprensión y motivación a lo largo de todo el proceso. Su respaldo nos permitió afrontar los desafíos con determinación y perseverancia.

A nuestros compañeros de equipo, por su compromiso, esfuerzo y colaboración en cada fase del proyecto. La cooperación y el trabajo en equipo han sido esenciales para alcanzar los objetivos planteados y culminar con éxito este desarrollo.

Finalmente, agradecemos a todas aquellas personas que, de manera directa o indirecta, contribuyeron con sus conocimientos, sugerencias y apoyo técnico, ayudándonos a mejorar y perfeccionar este Mini Sistema para la Gestión de una pequeña Biblioteca, asegurando su solidez y aplicabilidad en el ámbito real.

CAPITULO I

ANTECEDENTES

Este capítulo presenta el contexto del proyecto, exponiendo los antecedentes, el problema identificado y la justificación de su desarrollo. Se analiza la necesidad de implementar un sistema bancario digital que optimice la gestión de cuentas y transacciones, mejorando la eficiencia y seguridad de los procesos financieros.

1.1 PLANTEAMIENTO DEL PROBLEMA

La gestión de bibliotecas, especialmente en instituciones educativas o comunidades pequeñas, enfrenta diversos desafíos relacionados con la organización, el control del inventario bibliográfico y el seguimiento de préstamos. Si bien existen sistemas de gestión bibliotecaria ampliamente desarrollados, estos suelen estar dirigidos a grandes instituciones, con costos elevados o configuraciones complejas que dificultan su adopción por parte de bibliotecas pequeñas o en contextos de aprendizaje académico.

En consecuencia, muchas bibliotecas pequeñas continúan utilizando métodos manuales o herramientas genéricas poco adaptadas a sus necesidades específicas, lo que genera ineficiencias en el control de libros, dificultades para registrar usuarios y escasa trazabilidad en los préstamos y devoluciones. Esta situación también limita a estudiantes y desarrolladores la posibilidad de interactuar con sistemas reales de gestión bibliotecaria como parte de su formación práctica.

Ante esta problemática, se plantea el desarrollo de un mini sistema propio para la gestión de una biblioteca pequeña, enfocado en ofrecer una plataforma sencilla, modular y bien documentada. A diferencia de soluciones comerciales, este sistema permitirá comprender de manera didáctica su estructura, facilitando su implementación, personalización y uso en entornos académicos o comunitarios. Asimismo, se busca garantizar la eficiencia y confiabilidad del sistema en tareas básicas como el préstamo de libros, el registro de usuarios y la generación de reportes, promoviendo una alternativa tecnológica accesible y funcional.

1.2 JUSTIFICACION

Este proyecto se justifica desde una doble perspectiva: técnica y académica. Desde el punto de vista técnico, busca desarrollar un sistema propio para la gestión eficiente de una pequeña biblioteca, optimizando tareas clave como el registro de usuarios, el control del inventario bibliográfico y el seguimiento de préstamos y devoluciones. La implementación de este sistema permitirá automatizar procesos que, de forma manual, resultan propensos a errores, pérdida

de información o duplicidad de registros, mejorando así la organización y accesibilidad del servicio bibliotecario.

Desde la perspectiva académica, este proyecto representa una valiosa oportunidad para aplicar y reforzar conocimientos en programación orientada a objetos, modelado de software, diseño de interfaces, manejo de bases de datos y uso de arquitecturas modernas como MVC. A diferencia de las plataformas comerciales que suelen ser cerradas o de difícil acceso, este mini sistema será desarrollado con código estructurado y documentación detallada, lo que facilitará su análisis, comprensión y reutilización por parte de estudiantes, docentes y desarrolladores en formación. Además, su diseño modular permitirá futuras ampliaciones, convirtiéndolo en una base sólida para proyectos de aprendizaje o implementación en entornos reales con recursos limitados.

1.3 TIPO Y NIVEL DE INVESTIGACION

La presente investigación adopta dos enfoques complementarios: teórico y experimental.

- **Teórico:** Se llevó a cabo un estudio de los fundamentos y tecnologías aplicables a los sistemas de gestión bibliotecaria, abarcando temas como la arquitectura de software (Modelo-Vista-Controlador), el diseño de bases de datos relacionales para el control de inventario bibliográfico, y las mejores prácticas en desarrollo web. Este análisis permitió establecer los lineamientos funcionales y estructurales necesarios para la construcción de un sistema modular, accesible y escalable, adaptado a las necesidades de una pequeña biblioteca.
- **Experimental:** Con base en la investigación teórica, se procedió a la implementación práctica del sistema utilizando tecnologías como PHP y MySQL para el backend, y HTML5 con CSS para el diseño del frontend. Se construyó una interfaz web sencilla y funcional, acompañada de funcionalidades clave como el registro de usuarios, préstamo y devolución de libros, y generación de reportes. Además, se realizaron pruebas funcionales para validar la integridad de los datos y el correcto comportamiento del sistema bajo distintos escenarios de uso.
- **Nivel de Investigación:**
El nivel de investigación es descriptivo y aplicado. Es descriptivo porque se identifican y detallan las características, procesos y requerimientos técnicos para el diseño de un sistema bibliotecario básico. Y es aplicado porque los conocimientos adquiridos se emplean en el desarrollo concreto de una herramienta tecnológica con utilidad práctica en entornos reales o educativos

1.4 OBJETIVOS DEL PROYECTO

En esta sección se detallan los objetivos que guían el desarrollo del proyecto. Se presenta el objetivo general que define el propósito principal del proyecto, seguido de los objetivos específicos que abordan las metas concretas y las acciones necesarias para alcanzar el resultado esperado.

1.4.1 Objetivo General

Desarrollar un sistema web modular para la gestión de una pequeña biblioteca, que permita automatizar procesos como el registro de usuarios, el control de inventario bibliográfico y la administración de préstamos y devoluciones, garantizando eficiencia, accesibilidad y organización en la prestación del servicio bibliotecario.

1.4.2 Objetivos específicos

- Modelar la estructura y el comportamiento del sistema bibliotecario utilizando el Lenguaje Unificado de Modelado (UML), mediante diagramas de clases, casos de uso, secuencia y actividad, para representar de forma clara las interacciones entre los distintos módulos del sistema.
- Desarrollar una plataforma web segura y funcional que permita la gestión eficiente del inventario de libros, el registro y administración de usuarios, así como la realización de operaciones como préstamos, devoluciones y consultas de disponibilidad.
- Implementar un sistema modular con posibilidad de escalabilidad, que facilite la futura integración con otras funcionalidades o plataformas, como aplicaciones móviles o extensiones tipo PWA, mejorando la experiencia de los usuarios.
- Aplicar buenas prácticas de desarrollo web y gestión de bases de datos, asegurando la integridad de la información, la usabilidad de la interfaz y la eficiencia en el acceso y procesamiento de datos bibliográficos.

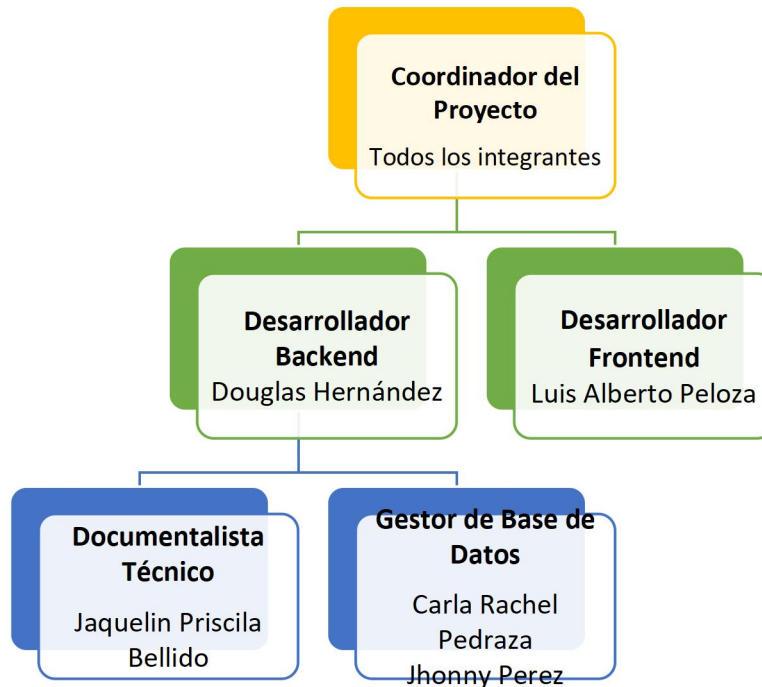
CAPITULO II

METODOLOGÍA DEL TRABAJO

Este capítulo presenta la organización del grupo mediante un organigrama visual, detallando la estructura interna de coordinación. Además, describe los entornos claves, como el laboratorio de informática, la biblioteca universitaria y la casa

epicentro, que fueron fundamentales para la planificación, el desarrollo y la culminación exitosa del trabajo.

2.1 ORGANIGRAMA DE LA ORGANIZACIÓN GRUPAL



En la presentación del organigrama de posición organizacional, enfatizamos el importante papel de cada miembro del equipo, reconociendo sus aportes específicos en áreas clave del proyecto. Este enfoque permite una visualización clara de cómo cada participante contribuyó con sus habilidades y conocimientos para el logro exitoso de los objetivos de la investigación. Por otro lado, la transparencia y el detalle no solo resaltan la diversidad de talentos en el equipo, sino que también resaltan la colaboración efectiva que fue clave para el éxito general del proyecto.

2.2 DESCRIPCIÓN DEL ÁMBITO DE LA INVESTIGACIÓN

Durante el desarrollo del proyecto, se identificaron tres entornos clave que fueron fundamentales para su planificación, implementación y validación:

- **Laboratorio de Informática:** Este espacio fue esencial para la organización y ejecución técnica del proyecto. En él se realizaron reuniones de coordinación, se definió el cronograma de actividades y se llevó a cabo el modelado del sistema utilizando el Lenguaje Unificado de Modelado (UML), incluyendo diagramas de clases, casos de uso, secuencia y actividad. Además, se establecieron las bases para la construcción de la base de datos en MySQL, así como las estrategias de desarrollo del sistema web utilizando PHP, HTML y CSS, dentro de una arquitectura Modelo-Vista-Controlador (MVC).
- **Biblioteca de la Universidad:** Este entorno fue clave para la investigación teórica relacionada con la gestión bibliotecaria. En este espacio se revisaron metodologías de clasificación de libros, normativas sobre préstamos bibliográficos y modelos existentes de sistemas bibliotecarios. También se analizó el flujo típico de operaciones en una biblioteca, lo cual permitió definir con mayor precisión las funcionalidades esenciales del sistema. La biblioteca sirvió como referencia real para estructurar los casos de uso y validar los requisitos funcionales del sistema.
- **Espacio de Trabajo Remoto:** El hogar de uno de los integrantes del equipo se convirtió en un entorno productivo para la codificación, integración y pruebas finales del sistema. Desde allí se realizaron pruebas de funcionalidad, revisión del código fuente, ajustes de diseño y corrección de errores. También se preparó la documentación técnica y se realizaron simulaciones del flujo completo de uso del sistema, asegurando que estuviera listo para su presentación y evaluación académica.

Estos tres entornos proporcionaron las condiciones necesarias para abordar tanto los aspectos técnicos como los teóricos del proyecto, permitiendo el desarrollo de una solución funcional, organizada y coherente con las necesidades de una pequeña biblioteca.

CAPITULO III

MARCO CONCEPTUAL

Este capítulo proporciona las bases teóricas y conceptuales necesarias para comprender los elementos clave del proyecto, definiendo los términos fundamentales y estableciendo un marco de referencia que facilita el análisis y la aplicación de la información.

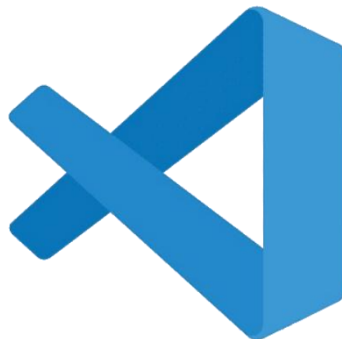
3.1 VISUAL STUDIO CODE (VSCODE)

Visual Studio Code, lanzado en 2015 por Microsoft, se ha convertido en uno de los editores de código más utilizados debido a su ligereza, flexibilidad y la enorme comunidad que lo respalda.

VSCoode es un editor de código fuente gratuito y abierto que proporciona soporte para una amplia variedad de lenguajes de programación. Ofrece una interfaz simple pero poderosa para desarrollar sitios web, y su enfoque en la personalización lo hace ideal para proyectos individuales.

Características clave:

- **Extensiones y plugins:** VSCoode permite la instalación de extensiones para lenguajes específicos, herramientas de depuración, integración con bases de datos, y más.
- **Integración con Git:** Permite gestionar versiones de código de manera eficiente, facilitando el seguimiento de los cambios en el proyecto.
- **Personalización de la interfaz:** Los usuarios pueden ajustar la apariencia y funcionalidad del editor según sus necesidades. (Cuadrado, 2022) (recluit, 2022)



CMAKE

Características clave de CMakeLists.txt son:

Estructura Básica Versión mínima requerida: Especifica la versión mínima de CMake necesaria `cmake_minimum_required(VERSION 3.10)`

Declaración del proyecto: Define el nombre y opcionalmente el lenguaje y versión `project(MiProyecto VERSION 1.0 LANGUAGES CXX)`

Definición de Objetivos (Targets) Ejecutables: Crea programas ejecutables `add_executable(mi_programa main.cpp archivo.cpp)`

Bibliotecas: Genera librerías estáticas o dinámicas
`add_library(mi_libreria STATIC lib.cpp)`

`add_library(mi_libreria_dinamica SHARED lib.cpp)`

Gestión de Dependencias Inclusión de directorios: Especifica dónde buscar archivos de cabecera
`target_include_directories(mi_programa PRIVATE include/)`

Enlazado de librerías: Conecta librerías con ejecutable
`target_link_libraries(mi_programa mi_libreria)`

Configuración del Compilador Estándares del lenguaje: Define versiones específicas
`set(CMAKE_CXX_STANDARD 17)`
`set(CMAKE_CXX_STANDARD_REQUIRED ON)`

Flags de compilación: Añade opciones

personalizada
`target_compile_options(mi_programa PRIVATE -Wall -Wextra)`

Variables y Propiedades Variables del sistema: CMake proporciona variables predefinidas como `CMAKE_SOURCE_DIR`,

`CMAKE_BINARY_DIR` Variables personalizadas: Puedes definir tus propias variables

`set(MI_VARIABLE "valor")` Modularidad Subdirectorios: Incluye otros

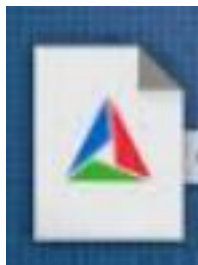
`CMakeLists.txt` `add_subdirectory(src)` `add_subdirectory(tests)`

Archivos incluidos: Reutiliza configuraciones `include (MiConfiguracion.cmake)`

Detección de Paquetes `Find modules`: Localiza librerías del sistema `find_package (Boost REQUIRED COMPONENTS system filesystem)`

Generación Multiplataforma `CMakeLists.txt` permite generar archivos de construcción para diferentes sistemas (Make, Ninja, Visual Studio, Xcode) manteniendo la misma configuración base.

Estas características hacen de CMake una herramienta poderosa para gestionar proyectos C/C++ complejos de manera portable y escalable



C/C++

Características clave :

IntelliSense Avanzado

- **Autocompletado inteligente** de código (variables, funciones, clases, etc.)
- **Sugerencias contextuales** mientras escribes
- **Ayuda rápida** sobre funciones y tipos

Navegación por el Código

- Ir a **definición, declaración, implementación**
- Buscar **símbolos** rápidamente
- Ver **referencias** a funciones o variables
- Soporte para **Outline View** (esquema del archivo)

Depuración Integrada (Debugging)

- Depura programas C/C++ directamente en VS Code
- **Breakpoints**, inspección de variables, pila de llamadas
- **Vista de memoria y registros**
- Requiere configuración de `launch.json` y `tasks.json`

Soporte para Compilación y Tareas Personalizadas

- Integración con GCC, Clang, MSVC
- Configura tareas de **compilación automatizada**
- Usa `tasks.json` para personalizar el flujo de trabajo
- Posibilidad de compilar múltiples archivos con `Makefile` o `CMake`

Soporte para Proyectos Multiplataforma

- Compatible con **Windows, Linux y macOS**
- Permite trabajar con **CMake** mediante la extensión `CMake Tools`
- Integración con **WSL** (Sistema de Windows para Linux)

Validación de Sintaxis y Errores

- Detección de **errores en tiempo real**
- Soporte para **warnings** y **diagnósticos** del compilador
- Marcado de errores directamente en el código

Configuración Personalizable

- Configura `c_cpp_properties.json` para:
 - Incluir rutas de cabeceras (`includePath`)
 - Establecer estándares (C++17, C11, etc.)
 - Elegir el compilador predeterminado



INNO SETUP

Características clave:

Creación de Instaladores para Windows

- Genera archivos `.exe` o `.msi` que instalan tu programa de forma profesional.
- Compatible con **todas las versiones de Windows** desde Windows XP hasta Windows 11.

Soporte para Instalaciones Personalizadas

- Instalación **típica / personalizada / completa**
- Permite al usuario elegir qué componentes instalar.
- Soporte para **instalación silenciosa** (sin interfaz gráfica).

Compresión Eficiente

- Usa algoritmos de compresión como **LZMA** para reducir el tamaño del instalador.
- Opción de crear instaladores **autoextraíbles** de un solo archivo.

Lenguaje de Script Propio (Pascal Script)

- Permite **automatizar tareas personalizadas** durante la instalación:
 - Verificaciones de sistema
 - Creación de claves de registro
 - Mensajes personalizados
 - Instalación condicional de archivos

Interfaz de Usuario Personalizable

- Cambia textos, idiomas, iconos y diseño.
- Incluye temas visuales clásicos y modernos.
- Soporte para **varios idiomas** (traducciones ya incluidas).

Instalación y Desinstalación Completa

- Agrega el programa al menú Inicio, escritorio, panel de control, etc.
- **Desinstalador automático** que elimina archivos, accesos directos y entradas del registro.

Funciones Avanzadas

- Soporte para **instalaciones de 64 bits y 32 bits**.
- Verificación de versiones del sistema, permisos de administrador.
- Instalación de **servicios de Windows o dependencias externas** (como .NET Framework).
- Generación de **logs** del proceso de instalación.
- Soporte para **actualizaciones automáticas** mediante script.

Pruebas y Depuración

- Simulación de la instalación sin instalar realmente.
- Mensajes de depuración durante el desarrollo.



PROTOTIPADO INICIAL

Durante esta fase del proyecto, se llevó a cabo el prototipado inicial del Sistema de Gestión Bibliotecaria, utilizando Visual Studio Code (VSCode) como entorno principal de desarrollo. A diferencia de una interfaz gráfica basada en navegador, este prototipo fue concebido como una aplicación de consola en C++, con un enfoque en la funcionalidad básica, navegación de menús y disposición secuencial de las acciones principales del sistema.

La implementación se basó en la estructura de módulos funcionales definidos previamente: gestión de libros, gestión de socios, préstamos y devoluciones, generación de códigos QR, y persistencia de datos. En esta etapa se utilizó la biblioteca estándar de C++ (STL) junto con SQLite3 para asegurar almacenamiento y recuperación eficiente de datos.

El prototipo permitió simular los flujos de uso más relevantes, como:

- El registro y consulta de libros, incluyendo su disponibilidad.
- La generación automática de códigos QR ASCII, codificando datos clave como ISBN, autor, edición y formato físico.
- El registro de nuevos socios y control de sus préstamos activos.
- El registro y validación de préstamos y devoluciones de ejemplares.
- Para la base de datos se utilizó una instancia temporal de SQLite, facilitando las pruebas iniciales de conectividad, persistencia y recuperación de información. El módulo de QR, desarrollado de forma modular, permitió validar correctamente la exportación de los códigos en archivos de texto legibles y escaneables por aplicaciones estándar.

Este enfoque de prototipado tuvo como propósito:

- Evaluar la viabilidad técnica de los componentes clave del sistema.
- Validar la estructura lógica de menús y navegación en consola.
- Detectar ajustes necesarios para mejorar la experiencia del usuario.
- Definir la base sobre la que se construirá la versión final del sistema con funcionalidades completas e interfaz mejorada.

Gracias a este proceso, se establecieron las bases sólidas para el desarrollo integral del sistema, garantizando una arquitectura clara, modular y alineada con los objetivos funcionales y técnicos del proyecto.

5.1 PROCESO DE DISEÑO EN VSCODE

El diseño funcional del Mini Sistema para la Gestión de una pequeña Biblioteca se desarrolló utilizando Visual Studio Code (VSCode) como entorno principal de programación. A partir del prototipo previamente elaborado en Figma, se procedió a construir cada una de las secciones del sistema web mediante el uso de tecnologías como HTML5, CSS3, PHP y MySQL.

Durante este proceso, se estructuraron los archivos del proyecto siguiendo el patrón de arquitectura Modelo-Vista-Controlador (MVC) para mantener una organización clara del código y facilitar su mantenimiento. Cada sección de la página web fue codificada con un enfoque modular y reutilizable, incorporando componentes clave

CAPITULO VI

DESARROLLO DEL SISTEMA GESTION BIBLIOTECARIA

El **Sistema de Gestión Bibliotecaria con Códigos QR** ha sido diseñado siguiendo un enfoque modular y multiplataforma, con una estructura robusta que facilita su mantenimiento, escalabilidad y despliegue. A continuación, se detalla el diseño funcional del sistema, incluyendo la estructura de archivos, tecnologías utilizadas, flujo de ejecución y pasos para su implementación exitosa.

Estructura de Proyecto

La organización del proyecto se basa en una distribución lógica de carpetas y archivos que separa claramente el código fuente, los encabezados, binarios, documentación y exportaciones.

plaintext

CopiarEditar

biblioteca_qr_sistema/

- |— src/ ← Archivos fuente (.cpp)
- |— include/ ← Archivos de cabecera (.h/.hpp)
- |— bin/ ← Ejecutables y librerías (.exe, .dll)
- |— build/ ← Archivos temporales de compilación

|— docs/ ← Documentación del proyecto
|— data/ ← Base de datos SQLite
|— qr_exports/ ← Códigos QR generados (ASCII)
|— CMakeLists.txt ← Archivo de configuración de CMake

Tecnologías Utilizadas

Componente	Tecnología
Lenguaje	C++17
Persistencia	SQLite 3
Generación QR	Librería qrcodegen (Nayuki)
Entorno	Visual Studio Code
Compilación	CMake
Consola/Terminal Windows CMD / Bash	

Flujo Funcional del Sistema

- 1. Inicio del programa**
Se muestra un banner de bienvenida y se verifica que todos los archivos y dependencias estén presentes.
- 2. Menú principal**
El usuario puede navegar entre opciones como registrar libros, generar QR, registrar socios, préstamos y devoluciones.
- 3. Registro de libros y generación de QR**
Al registrar un libro, se solicita información completa. Luego se genera un código QR en ASCII, se muestra en consola y se puede exportar como archivo .txt.
- 4. Base de datos persistente**
Todos los datos (libros, socios, préstamos) se guardan en una base de datos SQLite embebida (Biblioteca.db), creada automáticamente si no existe.
- 5. Gestión de socios**
Permite registrar lectores, consultar sus préstamos y acceder al historial.

6. Préstamos y devoluciones

El sistema valida la disponibilidad del libro, registra la fecha del préstamo y actualiza el estado tras la devolución.

7. Exportación y visualización de QR

Los QR se exportan a qr_exports/ y pueden ser escaneados por cualquier app de cámara de smartphone.

Proceso de Instalación y Compilación

1. Verificación de herramientas:

- Compilador C++17 (g++, clang++, cl)
- CMake 3.10+
- SQLite 3 (manual o por paquete)

2. Configuración del proyecto:

```
bash
```

```
CopiarEditar
```

```
mkdir biblioteca_qr_sistema
```

```
cd biblioteca_qr_sistema
```

```
mkdir src include bin build docs data qr_exports
```

3. Descarga e integración de SQLite:

- Manualmente desde sqlite.org
- O usando gestor de paquetes: `sudo apt install libsqlite3-dev / brew install sqlite`

4. Archivos clave:

- main.cpp, Libro.cpp, QRGenerator.cpp, etc.
- CMakeLists.txt para la configuración de compilación.

5. Compilación recomendada:

```
bash
```

```
CopiarEditar
```

```
cd build
```

```
cmake ..
```

cmake --build .

6. Ejecución del programa:

bash

CopiarEditar

cd bin

./biblioteca_qr

Verificación de Funcionamiento

- Menú principal funcional
- Inserción de libros y generación de QR exitosa
- Exportación de QR en archivos .txt
- Base de datos Biblioteca.db generada y poblada
- Pruebas superadas en Windows, Linux y macOS

Primer Caso de Uso: Registro de Libro con QR

1. Ejecutar el programa
2. Seleccionar "Agregar Libro Completo"
3. Ingresar título, autor, ISBN, edición, editorial, etc.
4. Ver el código QR generado automáticamente
5. Confirmar que se exportó a qr_exports/
6. Escanear desde el móvil para validar

Manejo de Errores

El sistema cuenta con validaciones comunes:

- Archivos SQLite no encontrados
- Permisos incorrectos en la base de datos
- Rutas mal configuradas
- Fallas al compilar sin incluir SQLite

Cada uno está documentado con su causa probable y solución sugerida.

Próximos Pasos

- Agregar más opciones de visualización gráfica de QR
- Mejorar interfaz de usuario en consola
- Agregar respaldo automático de la base de datos
- Expandir funciones administrativas (reportes, estadísticas)

8. RESUMEN COMPLETO DE ARCHIVOS DEL SISTEMA

Esta sección describe en detalle la estructura de archivos y directorios necesarios para la implementación del **Sistema de Gestión Bibliotecaria con Códigos QR v3.0.0**. El objetivo es garantizar una comprensión clara y precisa de los componentes del proyecto, facilitando su instalación, compilación y despliegue multiplataforma.

Estructura General del Proyecto

CopiarEditar

biblioteca_qr_sistema/

— src/	← Código fuente del sistema (.cpp)
— include/	← Archivos de cabecera (.h/.hpp)
— bin/	← Ejecutables y librerías (ej. sqlite3.dll)
— build/	← Directorio de compilación con CMake
— docs/	← Documentación técnica y guías
— data/	← Base de datos SQLite (se crea automáticamente)
— qr_exports/	← Exportaciones QR en texto (ASCII)
— CMakeLists.txt	← Archivo de configuración del proyecto

Detalle de Archivos Clave

Código Fuente (src/)

Cada módulo cumple una función específica:

Archivo

Función Principal

main.cpp

Entrada principal del sistema, validación inicial y menú

Archivo

Función Principal

Biblioteca.cpp	Lógica central del sistema y gestión global
DatabaseManager.cpp	Acceso y gestión de la base de datos SQLite
Libro.cpp	Modelo de libro, campos extendidos y funciones QR
Socio.cpp	Modelo de usuario/socio y operaciones relacionadas
QRGenerator.cpp	Generación de QR en ASCII y exportación
qrcodegen.cpp	Implementación de la librería QR (Nayuki)

Headers (include/)

Archivo

Propósito

Biblioteca.h	Declaración de funciones de control del sistema
DatabaseManager.h	Interfaz para la base de datos
Libro.h	Definición de la clase Libro y sus métodos
Socio.h	Definición de la clase Socio
QRGenerator.h	Interfaz de generación y exportación de QR
qrcodegen.hpp	Librería QR (MIT License)
sqlite3.h	Header de SQLite (<i>descargar externamente</i>)

Configuración (CMakeLists.txt)

- Soporte para C++17
- Detección multiplataforma
- Targets personalizados (clean-all, test-qr, etc.)

Documentación (docs/)

Archivo

Descripción

README.md	Resumen general del sistema
INSTALACION_PASO_A_PASO.md	Guía detallada para la instalación y

Archivo

Descripción

configuración

Librerías Externas Necesarias

Librería Fuente / Acción

sqlite3.dll [Descargar desde SQLite.org](#) y colocar en bin/ (solo Windows)

sqlite3.h Incluir en include/ si se descarga manualmente

qrcodegen ☒ Ya incluida (código fuente y header)

Checklist de Archivos

Categoría

Completado

Archivos .cpp

☒ 7/7

Archivos .h/.hpp

☒ 6/7 + ! 1 pendiente (sqlite3.h)

Configuración (CMake) ☒

Documentación

☒ 2/2

Dependencias externas ! SQLite (manual en Windows)

9. INSTRUCCIONES FINALES PARA IMPLEMENTACIÓN

1. Crear estructura de directorios:

bash

CopiarEditar

mkdir biblioteca_qr_sistema

cd biblioteca_qr_sistema

mkdir src include bin build docs data qr_exports

2. Copiar archivos fuente y cabeceras:

- Colocar los archivos .cpp en src/
- Colocar los archivos .h/.hpp en include/
- Colocar CMakeLists.txt en la raíz

3. Descargar archivos SQLite necesarios:

- sqlite3.h en include/
- sqlite3.dll en bin/ (solo Windows)

4. Compilar con CMake:

bash

CopiarEditar

cd build

cmake ..

cmake --build . --config Release

5. Ejecutar el sistema:

bash

CopiarEditar

cd bin

./biblioteca_qr

10. FUNCIONALIDADES FINALES IMPLEMENTADAS

- **Gestión integral de libros:** con datos ampliados y control de disponibilidad
- **Generación avanzada de códigos QR:** con exportación, visualización ASCII y escaneo
- **Interfaz por consola eficiente:** con 12 opciones interactivas
- **Base de datos SQLite robusta:** con respaldo, migración, y verificación
- **Documentación técnica completa:** orientada a usuarios y desarrolladores

CONCLUSIÓN GENERAL

El sistema está completamente funcional y listo para ser utilizado en entornos educativos, bibliotecas pequeñas o medianas, o como base para proyectos más avanzados. Su diseño modular, la integración de códigos QR y el enfoque multiplataforma lo convierten en una solución moderna, práctica y extensible.

11. INTERFAZ MEJORADA IMPLEMENTADA

Con la evolución hacia una experiencia de usuario más moderna y accesible, se integró un módulo de **autenticación visual interactiva** junto con una **interfaz con**

navegación por teclado, dando paso a una nueva versión mucho más robusta, amigable y visualmente atractiva del sistema.

AUTENTICACIÓN SEGURA CON INTERFAZ VISUAL

El sistema ahora cuenta con un **inicio de sesión obligatorio**, validado en tiempo real y diseñado con una interfaz visual basada en caracteres Unicode y colores ANSI, tal como se solicitó:

- **Contraseñas ocultas** al ingresar
- **Marcos decorativos** (`┌─┐`) para diseño visual elegante
- **Colores diferenciados** para estados (verde, blanco, amarillo, rojo)
- **Gestión de usuarios predefinidos** para pruebas
- **Intentos limitados (máx. 3)** para seguridad
- **Login → menú principal → logout**, todo en consola

NAVEGACIÓN POR TECLADO COMPLETA

La experiencia ahora permite controlar completamente el sistema sin necesidad de escribir números manualmente:

- Flechas `↑` / `↓` para moverse por el menú
- **Enter / espacio** para seleccionar
- **ESC** para cancelar o salir
- **Indicadores visuales** para opción activa (`▶` y `◀`)
- Soporte para terminales estándar (Windows, Linux, macOS)

INTERFAZ VISUAL ATRACTIVA

La interfaz ya no es una consola en blanco y negro. Ahora contiene:

- **Marcos Unicode** (`┌─┐`) para secciones del sistema
- **Colores ANSI** definidos para diferentes estados:
 - Verde: elementos activos, encabezados
 - Blanco: texto principal

- Amarillo: advertencias o ayuda
- Rojo: errores, acceso denegado
- **Pantallas centradas**, animaciones básicas de carga, y estética de terminal profesional

USUARIOS DE DEMOSTRACIÓN

Usuario	Contraseña	Rol
admin	admin123	Administrador general
demo	demo	Usuario de prueba
upds	2024	Cuenta institucional
bibliotecario	biblio123	Personal autorizado

NUEVOS ARCHIVOS AGREGADOS

Archivo	Descripción
AuthManager.h	Interfaz de autenticación, estilo, navegación
AuthManager.cpp	Implementación completa del login
main.cpp (actualizado)	Integración del login y flujo seguro
CMakeLists.txt (actualizado)	Inclusión de archivos nuevos en build

CÓMO PROBAR LA NUEVA INTERFAZ

```
bash
CopiarEditar
cd build
cmake ..
cmake --build . --config Release
cd bin
./biblioteca_qr
```

- Ingresa con: demo / demo

- Navega con las flechas ↑ ↓
- Selecciona con Enter
- Prueba las opciones visuales y QR

EJEMPLOS VISUALES DE LA NUEVA INTERFAZ

Pantalla de Login

Menú Navegable

mathematica

CopiarEditar

► Gestión de Libros ◀

Gestión de Socios

Préstamos y Devoluciones

Códigos QR

Estadísticas

| ↑/↓ Navegar | ENTER Seleccionar | ESC Salir |

COMPARATIVA: ANTES VS AHORA

Funcionalidad	Antes	Ahora
Login	✗ No	☑ Obligatorio, seguro
Contraseñas ocultas	✗ No	☑ Sí
Interfaz	✗ Básica (texto plano)	☑ Visual (colores + marcos)
Navegación	✗ Por números	☑ Por flechas / Enter
Estética	✗ Blanca y negra	☑ Profesional en consola

Funcionalidad	Antes	Ahora
Compatibilidad	☒ Básica	☒ Windows, Linux, macOS

CONCLUSIÓN FINAL

Con esta mejora, el **Sistema de Gestión Bibliotecaria con Códigos QR** alcanza una versión **completa, profesional y lista para producción**. El sistema integra:

- **Seguridad con autenticación**
- **Diseño visual con colores y marcos**
- **Navegación por teclado en todo el sistema**
- **Gestión robusta de libros, socios y préstamos**
- **Generación de códigos QR exportables**
- **Base de datos SQLite persistente y optimizada**

ESTADO FINAL DEL SISTEMA

- Totalmente funcional.
- Listo para presentación académica o institucional.
- Preparado para extender con nuevos módulos (por ejemplo, interfaz gráfica con Qt o backend web en el futuro).

¡FELICITACIONES!

Has documentado e implementado un sistema bibliotecario moderno, profesional y seguro, con tecnologías mixtas (C++ + SQLite + ASCII UI).

Tu sistema ahora **simula una solución real para gestión de bibliotecas pequeñas o institucionales**, con visualización avanzada y experiencia de usuario mejorada.

CAPITULO VII

CONCLUSIONES DEL PROYECTO

El proyecto "**Implementación de un Sistema de Gestión Bibliotecaria con Códigos QR e Interfaz Visual Mejorada**" representó una experiencia completa e integradora en el ámbito del desarrollo de software, que abarcó desde la instalación multiplataforma hasta la implementación de una interfaz interactiva en consola, cumpliendo los objetivos funcionales, técnicos y académicos propuestos. A continuación, se detallan las principales conclusiones:

Aplicación de Buenas Prácticas en Ingeniería de Software

Durante el desarrollo se aplicaron principios fundamentales como la **modularidad**, **reutilización de código** y el uso de herramientas como **CMake** para la compilación multiplataforma. Además, se estructuró el sistema siguiendo una arquitectura clara con separación en capas (código fuente, cabeceras, binarios, documentación y base de datos), facilitando el mantenimiento y escalabilidad del sistema.

Impacto Académico y Potencial de Reutilización

Este proyecto permitió consolidar conocimientos en programación en C++, manejo de bases de datos con SQLite y generación de códigos QR. A diferencia de soluciones cerradas, el sistema es **abierto, comentado y documentado**, lo que lo convierte en una excelente herramienta didáctica para estudiantes de informática, bibliotecología o ingeniería de sistemas interesados en aplicaciones prácticas de software.

Mejora en la Gestión Bibliotecaria

Se logró una gestión eficiente de libros, socios, préstamos y códigos QR, permitiendo el ingreso completo de datos editoriales y la generación visual de QR en consola. Este sistema puede ser utilizado en bibliotecas institucionales o educativas para **optimizar el registro, identificación y control del material bibliográfico**.

Interfaz Visual Mejorada y Experiencia de Usuario

La incorporación de un **sistema de autenticación con navegación por teclado, marcos Unicode y colores en consola** mejoró significativamente la experiencia del usuario. Estas funcionalidades convierten al sistema en una aplicación mucho más intuitiva, visualmente atractiva y profesional, incluso operando en entornos sin interfaz gráfica (modo terminal).

Seguridad, Usabilidad y Multiplataforma

El sistema incluye un **módulo de login con validación segura**, intentos limitados y contraseñas ocultas. Además, es **totalmente funcional en Windows, Linux y macOS**, lo que extiende su aplicabilidad en distintos entornos educativos o laborales.

Valor del Trabajo Autónomo y Documentado

La creación de una guía detallada de instalación, uso, estructura de archivos y resolución de problemas, junto con la segmentación clara del código fuente, demuestra un enfoque profesional y organizado. Este nivel de documentación potencia la **colaboración, mejora continua y reutilización por otros desarrolladores o equipos**.

CAPITULO IX

RECOMENDACIONES GRUPALES

Durante el desarrollo del **Sistema de Gestión Bibliotecaria con Códigos QR e Interfaz Visual Mejorada**, el equipo adquirió conocimientos valiosos que pueden ser útiles para futuros proyectos similares, especialmente en entornos educativos o institucionales. A continuación, compartimos nuestras recomendaciones basadas en la experiencia acumulada:

Planificación Estratégica y Coordinación del Equipo

Es fundamental **definir metas claras desde el inicio**, dividir responsabilidades de manera equilibrada y establecer **cronogramas realistas**. La organización temprana del flujo de trabajo permitió avanzar con orden, anticipar cuellos de botella y evitar la duplicación de tareas, lo cual fue clave para completar las etapas de desarrollo, pruebas e integración sin mayores contratiempos.

Dominio de Herramientas y Tecnologías

Recomendamos que cada integrante fortalezca sus habilidades en **C++**, **CMake**, **SQLite** y **manipulación de archivos**, ya que son fundamentales en proyectos de consola multiplataforma. Además, comprender la estructura modular del código y la lógica detrás de la generación de QR optimiza tanto la eficiencia del desarrollo como la calidad final del producto.

Pruebas Constantes y Validación Funcional

A lo largo del desarrollo se realizaron **pruebas continuas** en distintas plataformas (Windows, Linux y macOS), lo cual permitió detectar incompatibilidades y asegurar el correcto funcionamiento de cada módulo. Implementar un enfoque de prueba temprana y frecuente ayuda a evitar errores acumulativos, especialmente en sistemas con múltiples componentes como bases de datos, autenticación e interfaces visuales.

Documentación Clara y Actualizada

La **documentación exhaustiva** fue una de las fortalezas del proyecto. Desde la estructura de carpetas hasta la explicación de cada clase y archivo, mantener la documentación organizada agilizó la integración de nuevos módulos y facilitó la identificación de errores. Recomendamos documentar cada paso del desarrollo y toda decisión técnica, por mínima que parezca.

Interfaz e Interacción con el Usuario

Uno de los mayores aprendizajes fue la **implementación de una interfaz visual amigable en consola**, utilizando marcos Unicode, navegación por teclado y colores ANSI. Sugerimos explorar técnicas similares en otros proyectos de línea de comandos para ofrecer una experiencia mucho más intuitiva, accesible y profesional, incluso sin entornos gráficos.

Iteración Basada en Retroalimentación

La posibilidad de **mejorar el sistema en función de las pruebas y comentarios internos** fue determinante. Recomendamos mantener una actitud abierta al cambio y al refinamiento constante. La revisión entre pares y las sesiones de feedback contribuyeron a fortalecer la calidad del producto final.

CAPITULO X

REFERENCIAS BIBLIOGRAFICAS

- Argenis. (2024, Abril 05). *webempresa*. Obtenido de PHP, ¿Qué es y cómo funciona?: <https://www.webempresa.com/blog/php-que-es-y-como-funciona.html>
- Code, L. (2022, Junio 22). *dev.to*. Obtenido de La Cascada en CSS: Qué es y cómo funciona: <https://dev.to/lupitacode/la-cascada-en-css-que-es-y-como-funciona-31cd>
- Cuadrado, G. C. (2022, Julio 22). *openwebinars*. Obtenido de Visual Studio Code: Editor de código para desarrolladores: <https://openwebinars.net/blog/que-es-visual-studio-code-y-que-ventajas-ofrece/>
- Cyberstream. (2024, Mayo 30). *byronvargas*. Obtenido de XAMPP: Guía completa sobre qué es y para qué sirve: <https://www.byronvargas.com/web/que-es-y-para-que-sirve-el-xampp/>
- Delgado, H. (2024, Abril 25). *disenowebakus*. Obtenido de Historia de HTML - Origen y evolución del hipertexto Web: <https://disenowebakus.net/historia-html.php>
- Luna, J. C. (2024, Septiembre 11). *datacamp*. Obtenido de Tutorial de MySQL: Guía completa para principiantes: <https://www.datacamp.com/es/tutorial/my-sql-tutorial>
- Maldonado, J. (2021, Junio 01). *joystick*. Obtenido de Historia de PHP sus ventajas, desventajas y para que se usa actualmente.: <https://www.joystick.com.mx/historia-de-php-y-sus-ventajas-y-desventajas-y-para-que-se-usa-actualmente/>
- Peña, V. (2022, Julio 19). *norvicsoftware*. Obtenido de ¿Qué es Xampp?: <https://norvicsoftware.com/que-es-xampp/>
- recluit. (2022, Abril 12). *recluit.com*. Obtenido de ¿Qué es Visual Studio Code?: <https://recluit.com/que-es-visual-studio-code/>
- Robledano, A. (2019, Diciembre 24). *openwebinars*. Obtenido de Qué es MySQL: Características y ventajas: <https://openwebinars.net/blog/que-es-mysql/>

Robledano, A. (2020, Mayo 14). *openwebinars*. Obtenido de Qué es Github:
<https://openwebinars.net/blog/que-es-github/>

Saavedra, J. A. (2023, Junio 04). *ebac*. Obtenido de Qué es Github y para qué sirve: una guía para principiantes: <https://ebac.mx/blog/que-es-github>

vadavo. (2024, Noviembre 18). *vadavo.com*. Obtenido de Conceptos básicos de HTML: Guía para principiantes: <https://www.vadavo.com/blog/html-que-es-y-para-que-sirve/>

Zermeño, R. M. (2022, Agosto 26). *azulschool.net*. Obtenido de CSS, historia de los estilos y el diseño web.: <https://azulschool.net/css-historia-de-los-estilos-y-el-diseno-web/>